I102 - Paradigmas de Programación

MANEJO DE MEMORIA

3

Prof. Mariano Scaramal

Ingeniería en Inteligencia Artificial

Que conceptos de C++?

- Conceptos:
  - La Memoria en la Ejecución de un Programa
  - Repaso de punteros en C
  - Punteros en C++ - Smart Pointers
  - Pasaje de Punteros a Funciones
  - Punteros a Funciones
  - Ejemplos con Callbacks
  - Funciones Lambda

Scope o Alcance

- Una variable podrá ser utilizada o existe dentro de un scope.
- Una variable declarada en una función, tendrá un <u>scope local</u> que va desde su declaración hasta el fin de la función.
- También, una variable puede tener un bloque de alcance (o block scope) si se la declara dentro de un bloque "{ … }".

Comienza el scope de x →

Finaliza el scope de x →

```
void myFunc(){
    double y = 5.9;
    int x = 2;
    ...
}
```

```
void myFunc(){
    double x = 5.9;
    {
        int x = 2;
        std::cout << x << std::endl;
    }
    std::cout << x << std::endl;
} // Primero imprimirá 2 y luego 5.9
```

Scope o Alcance Local / Global

```
#include <iostream>

void f() {
    int x = 5;
    std::cout << "Dentro de la
función f(): " << x << std::endl;
}

int main() {
    int x = 15;
    f();
    std::cout << "En main(): " << x
<< std::endl;
    return 0;
}
```

```
#include <iostream>
int x = 5; //Variable global
void f() {
    std::cout << "Dentro de la
función f(): " << x << std::endl;
}

int main() {
    f();
    std::cout << "En main(): " << x
<< std::endl;
    return 0;
}
```

La Memoria en la Ejecución de un Programa

- La memoria se divide en distintos segmentos:
  - Segmento de Código: contiene el código a ejecutar, es una memoria de sólo lectura y su tiempo de vida es largo (el del programa).
  - Segmento de Datos: contiene las ***global*** y ***static*** variables (las variables globales y estáticas). Esta es una memoria de lectura y escritura con variables cuya duración o tiempo de vida es largo.
  - Memoria Stack: contiene variables locales y los marcos de las funciones, con información necesaria para ejecutar la función: parámetros, dirección de retorno y variables. Memoria de lectura/escritura y de corta duración.
  - Memoria Heap: utilizada para almacenar variables requeridas dinámicamente mediante punteros o funciones como *new*, *malloc, etc.* Esta es una memoria de lectura/escritura, de corta duración y es manejada manualmente.

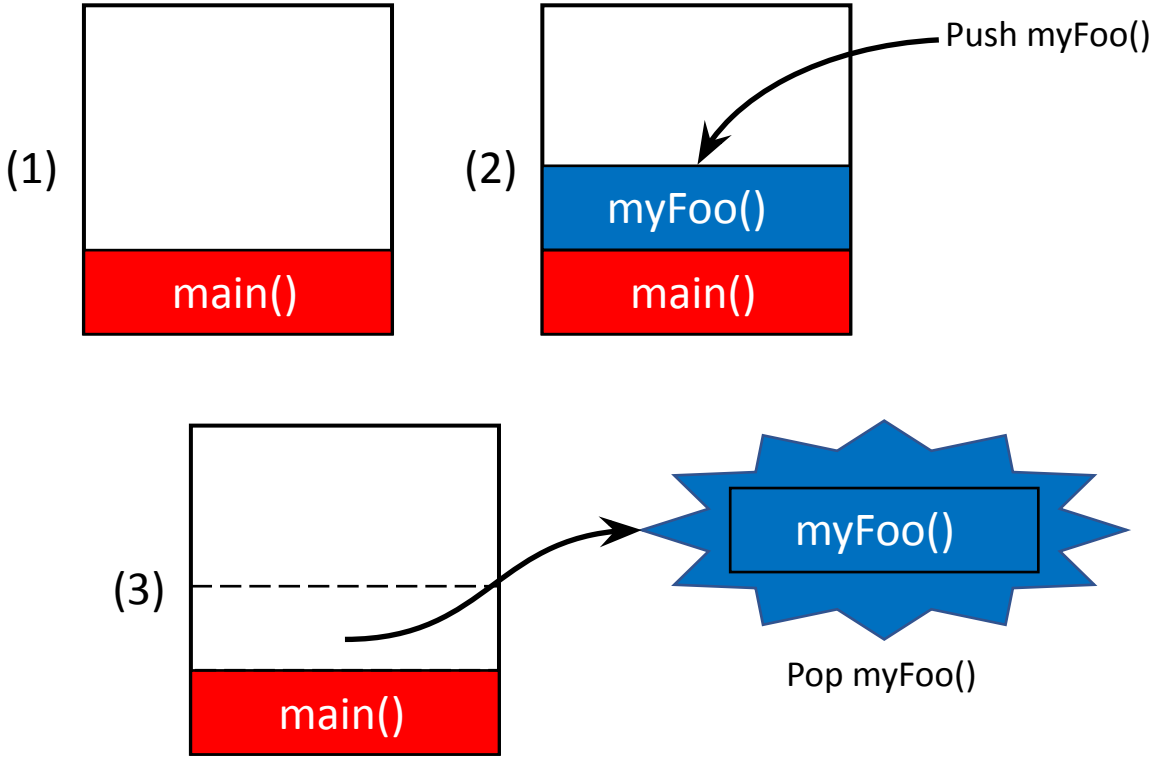
El Segmento de Datos

- Utilizadas al definir una variable tipo *global* o *static*. Esta memoria se divide en dos partes, una de estas partes es de sólo lectura y almacena constantes.
- Las variables *static* retienen sus valores entre llamadas a funciones.
- Al mismo tiempo, las variables estáticas sólo se pueden inicializar una vez.

```
void myFunction() {
    static int contador;            // Inicializada con cero una vez,
                                    // se usa la misma variable que ya tiene un valor.
    contador++;                     // La variable se incremente en cada llamada a la función.
    std::cout << "Contador: " << contador << std::endl;
}

int main() {
    myFunction();  // Imprime: Contador: 1
    myFunction();  // Imprime: Contador: 2
    myFunction();  // Imprime: Contador: 3
    return 0;
}
```

La Memoria Stack

- Está implementada como una pila, FIFO (First In, First Out).
- Tiene un límite en su tamaño y este no se puede cambiar.
- Este límite puede producir un stack overflow (recursión sin caso base).



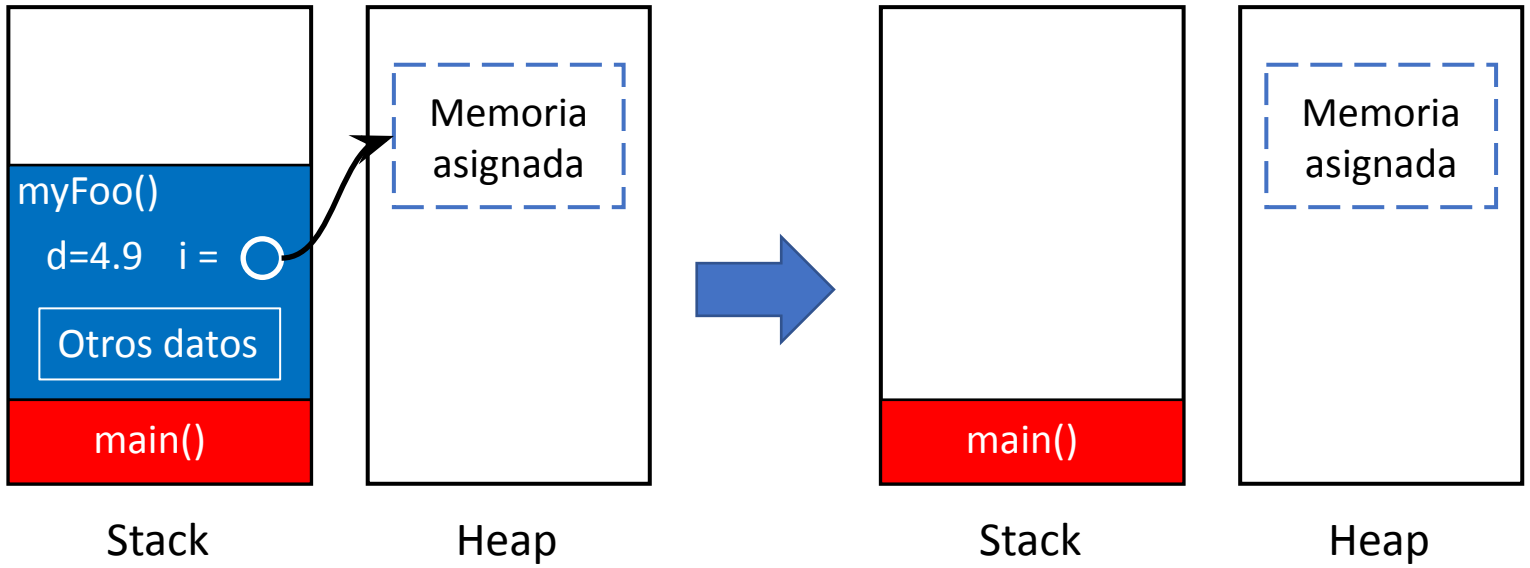

```
double myFoo(double d){
    std::cout << d << std::endl;
    return d/2;
}


int main(){
    int i = 2;
    double d = 5.1;
    myFunc(d);

    return 0;
}
```

(3) (1) (2) (1)

Uso de la memoria stack

La Memoria Heap

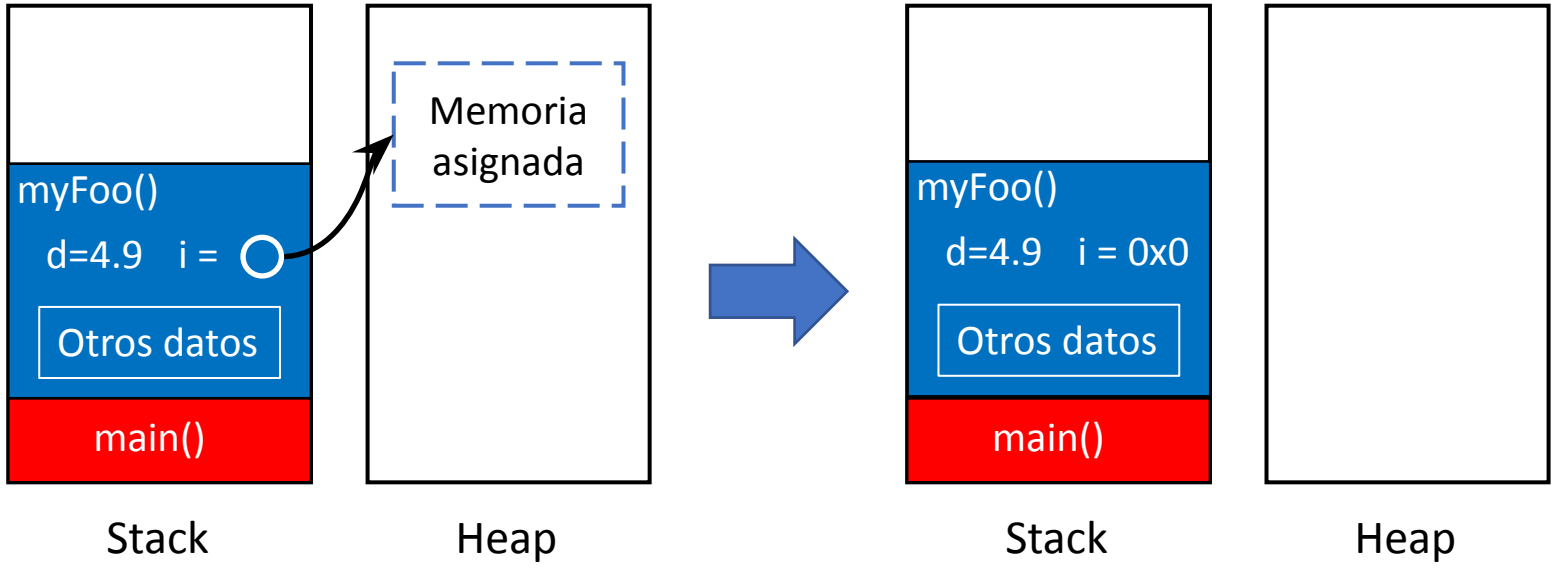
- Se utiliza con punteros o requiriendo memoria: *malloc, calloc, realloc* y *new*.
- Se debe liberar la memoria utilizando: *free* o *delete (C++).*
- Su tamaño es variable y se puede acceder en forma global.
- Si la memoria no se libera, quedará asignada y no se podrá reutilizar. A esto se lo conoce como *memory leak*.




```
double myFoo(){
   double d = 4.9;
   int *i = new int;
   *i = 1;
   return d;
}

int main(){
   double d = myFoo();
   return 0;
}
```

La Memoria Heap (cont'd)

- No liberar la memoria lleva a un *memory leak* que degrada la performance de los programas en ejecución, debido a la falta de memoria disponible.
- El sólo liberar la memoria genera un ***dangling pointer***, un puntero que apunta a un objeto que ha sido borrado. Esto puede generar distintos errores, hacer que el programa se caiga o que funcione bien cuando en realidad no lo hace.
- Apuntar a *nullptr* no resuelve el problema, pero generará un ***runtime error***.




```
double myFoo(){
    double d = 4.9;
    int *i = new int;
    *i = 1;
    delete i;
    i = nullptr;
    return d;
}
int main(){
    double d = myFoo();
    return 0;
}
```

Punteros C/C++

- C++ puede utilizar punteros definidos como en C.
- Permiten gestión eficiente de la memoria.
- Son variables que almacenan direcciones de memoria
- Se pueden liberar con la función *free*.

```
#include <iostream>

int main(){
   int *p1 = 0;          // Declaración e inicialización a cero
   int *p2 = NULL;       // Declaración null pointer
   int *p3 = nullptr;    // Declaración null pointer
   int *p4;              // Declaración sin inicializar
   int *p5 = (int*)malloc(100*sizeof(int));

   free(p1);  free(p2);  free(p3);  free(p4); free(p5);

   return 0;
}
```

Punteros C++

- Para el caso exclusivo de C++ el definir un raw pointer se hace con:

```
double *ptrD = new double;
int* ptrI = new int[100];
```

  - A diferencia de *malloc, new* no requiere un cast, por lo que es type safety.
  - Los valores del array son inicializados automáticamente con cero.
  - Si se utiliza con objetos, *new* llamará al constructor default del mismo.

- Para liberar la memoria, en C++ se utiliza la palabra reservada *delete*:

```
delete pointer;
delete[] ptrI;
```

  - El comando *delete* es usado en conjunto con el comando *new*.
  - En el caso de objetos, *delete* llama al destructor del objeto.
  - Recordar: un puntero es una dirección con tipo.

Punteros C++ - Ejemplos

- Ejemplos del uso de punteros.

- Si la memoria no se libera, también generará memory leaks.

```
#include <iostream>

int main(){
   // Inicializar contenido con 3
   int *iP = new int(3);
   // Inicializar contenido con 8.3
   float *fP = new float(8.3);
   // Inicializar array con contenido default
   int *iArrP1 = new int[30];
   // Inicializar contenido con 5, 3, 1 de un array de 3 elementos
   int *iArrP2 = new int[3]{5, 3, 1};

   delete iP; delete fP;
   delete[] iArrP1; delete[] iArrP2;

   return 0;
}
```

Punteros C++ - Ejemplos

```
#include <iostream>

int *f() {
    int x = 10; // variable local
    return &x;  // devolvemos su dirección
}

int main() {
    int *p = f();
    std::cout << *p << std::endl;
}
```

¿Qué problema tiene?

Aritmética de Punteros

Operaciones Permitidas
Solo un conjunto muy especifico de operaciones matemáticas está permitido con punteros:

Sumar/Restar un entero (ptr + N o ptr - N): Avanza o retrocede el puntero N cantidad de elementos en la memoria.

Incremento/Decremento (ptr++ o ptr--): Es la forma más común de iterar. Mueve el puntero exactamente al elemento siguiente o anterior.

Restar dos punteros (ptr2 - ptr1): Siempre y cuando ambos punteros apunten a elementos del mismo arreglo, esta resta no te devuelve la diferencia en bytes, sino la distancia en cantidad de elementos que hay entre ellos.

Sumar/Restar un entero

(ptr + N o ptr - N): Avanza o retrocede el puntero N cantidad de elementos en la memoria.

Incremento/Decremento

(ptr++ o ptr--): Es la forma más común de iterar. Mueve el puntero exactamente al elemento siguiente o anterior.

Restar dos punteros

(ptr2 - ptr1): Siempre y cuando ambos punteros apunten a elementos del mismo arreglo, esta resta no te devuelve la diferencia en bytes, sino la distancia en cantidad de elementos que hay entre ellos.

Aritmética de Punteros

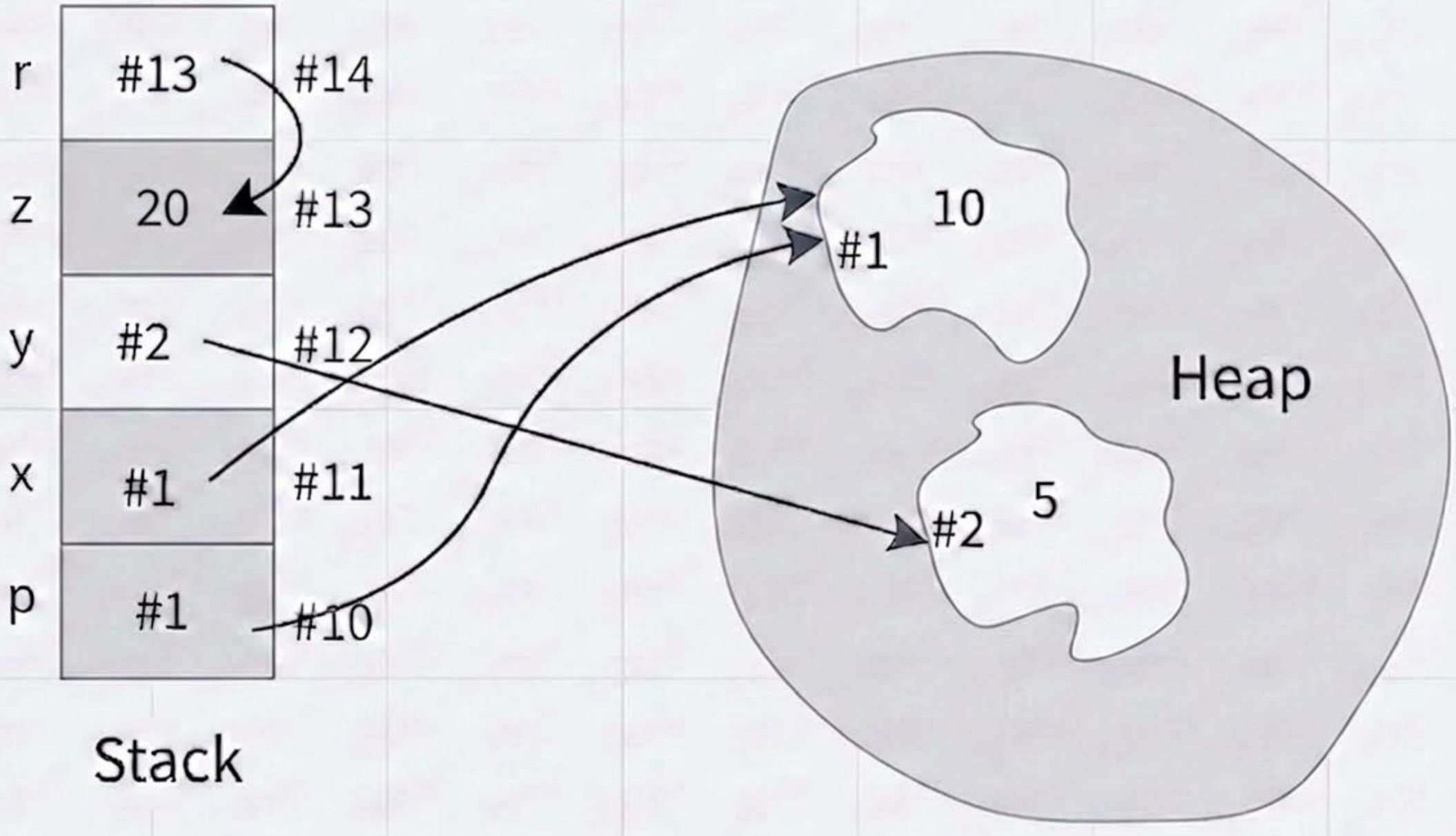
```
#include <iostream>


int main()
{
    int v[4] = {10, 20, 30, 40};
    int *p = v;
    std::cout << *(p + 2) << std::endl; // Imprime: ?
}
```

Aritmética de Punteros

- Se observa que los arrays en C++ se pasan a funciones como en C.
- Se pasa el nombre del array que contiene la dirección de su primer elemento.
- En el ejemplo, se imprimirá el contenido del array en dos formas distintas, dando:

10	-	10
20	-	20
30	-	30
40	-	40
50	-	50

```
#include <iostream>

void imprimirArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << "   -   "; // Elementos mediante índice
        // Elementos con aritmética de punteros
        std::cout << *(arr+i) << std::endl;
    }
    std::cout << std::endl;
}

int main() {
    int vals[] = {10, 20, 30, 40, 50};
    // Calcula el tamaño del array
    int size = sizeof(vals) / sizeof(vals[0]);
    // El nombre del array es la dirección de su primer elemento
    imprimirArray(vals, size);

    return 0;
}
```


r
#13
#14
z
20
#13
y
#2
#12
x
#1
#11
p
#1
#10
Stack
10
#1
Heap
5
#2

Smart Pointers (C++)

- En C++ moderno (en C++11) se introdujeron los smart pointers:

  #include <memory>

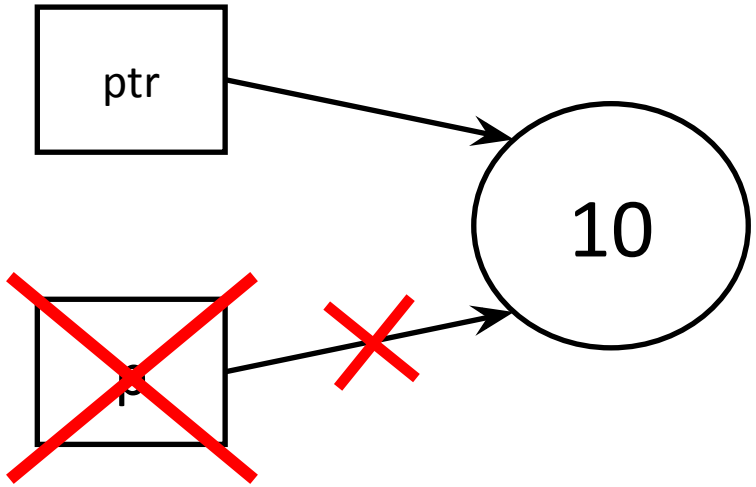
- Los mismos son clases con templates (se verá más adelante) que permiten eliminar la necesidad de liberar la memoria, evitando memory leaks.
- Hay 3 tipos de Smart Pointers, pero los más utilizados son:
  1. *unique_ptr<type>*
  2. *shared_ptr<type>*
- Estos punteros pueden definirse como:
  - *std::unique_ptr<int> ptr1 = std::make_unique<int>(3);*
  - *std::shared_ptr<int> ptr2 = std::make_shared<int>(4);*

Smart Pointers – unique_ptr

- Al utilizar un *unique_ptr*, el puntero **no puede compartirse**.

  std::unique_ptr<int> p (new int{10});
  std::unique_ptr<int> p2 = p; // Error!

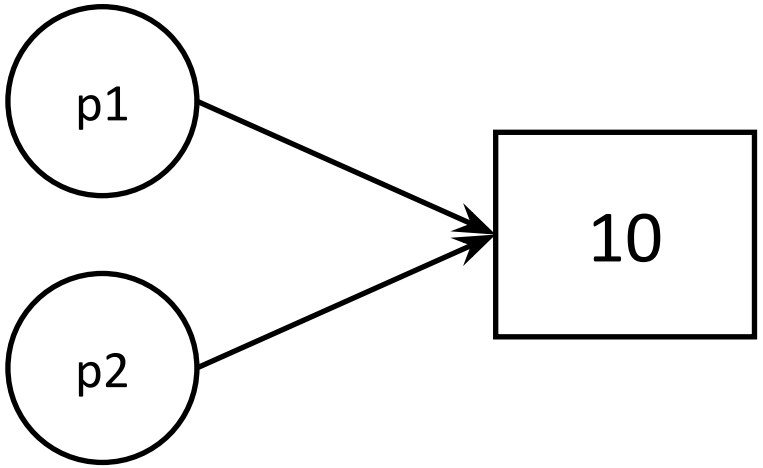
- Se puede cambiar de ownership en un *unique_ptr*. Esto se logra con *std::move* y hacer esto liberará la memoria del puntero original.




```
#include <iostream>
#include <memory>

void myFunc() {
    // Se crea el puntero
    std::unique_ptr<int> p (new int{10});
    // Cambia el ownership de p a ptr
    std::unique_ptr<int> ptr = std::move(p);
    // Imprime 10
    std::cout << *ptr << std::endl;
    // Imprime 10
    std::cout << *ptr.get() << std::endl;
    // Imprime la dirección de memoria
    std::cout << ptr.get() << std::endl;
} // Se liberara la memoria automáticamente al llegar al }

int main() {
    myFunc();
    return 0;
}
```

Smart Pointers – shared_ptr

- Los punteros *shared_ptr*, pueden compartir direcciones.
- Para ello, existe un contador que indica la cantidad de punteros.
- La posición de memoria será liberada cuando todos los punteros salen de scope.




```
#include <iostream>
#include <memory>

void myFunc() {
   std::shared_ptr<int> ptr1 = std::make_shared<int>(10);
   { // ptr2 comparte la dirección con ptr1
      std::shared_ptr<int> ptr2 = ptr1;
      std::cout << *ptr2 << std::endl;                // Imprime 10
      std::cout << ptr1.use_count() << std::endl;  // Imprime 2
   } // Solo ptr2 sale de scope, ptr2 se elimina, pero no ptr1
  std::cout << ptr1.use_count() << std::endl;  // Imprime 1
} // La memoria es liberada al salir ptr1 de scope

int main() {
   myFunc();

   return 0;
}
```

Smart Pointers – shared_ptr

- Problema con shared_ptr: la dependencia cíclica.
- Los punteros *shared_ptr incrementan el contador a 2*:
  - La primera al declarar los structs,
  - La segunda con los punteros a y b.
- Cuando el o los shared pointers van a salir de scope, observan que el contador no es 1 y no liberan la memoria.
- Esta dependencia entre los punteros se la denomina ***Dependencia Cíclica***. Esto produce memory leaks.

```
struct A;
struct B {
   std::shared_ptr<A> ptrA;  // B tiene un shared_ptr a A
   ~B() { std::cout << "B es liberado\n"; }};
struct A {
   std::shared_ptr<B> ptrB;  // A tiene un shared_ptr a B
   ~A() { std::cout << "A es liberado\n"; }};

int main() {
   std::shared_ptr<A> a = std::make_shared<A>();
   std::shared_ptr<B> b = std::make_shared<B>();
   // Se crea la referencia ciclica:
   a->ptrB = b;  b->ptrA = a;
   // Los punteros de A y B no se liberan porque aun
   // existen dos referencias, los count de a y b son 2.
   std::cout << "El use_count de a es: " <<
       a.use_count() << std::endl; // Imprime 2
   std::cout << "El use_count de b es: " <<
       b.use_count() << std::endl; // Imprime 2

   return 0;
} // No se ejecutan ~A() y ~B() porque los count > 1
```

Matriz de Smart Pointers

- Para crear un array 1D, se puede pasar la longitud del array en paréntesis.
- Para el 2D, se crea un array de arrays. Para ello, se crea un array de 2 posiciones de *unique_ptr*, inicialmente vacío, pero que contendrá punteros a arrays de a int.
- Luego, se asignan los punteros a los arrays para las dos posiciones/filas.
- Finalmente se recorre la matriz y se imprime su contenido (0 a 4).

```
#include <iostream>
#include <memory>
int main ()
{   // Crear un array de 10 unique pointers
    std::unique_ptr<int[]> arr1D = std::make_unique<int[]>(10);
  int filas = 2, columnas = 5;
   // Crear un array de 2 unique pointers, cada uno inicialmente
   // vacío, que luego podrá almacenar un array de 5 elementos.
    std::unique_ptr<std::unique_ptr<int[]>[]> arr2D =
std::make_unique<std::unique_ptr<int[]>[]>(filas);

    for ( int i = 0; i < 2; i++ )
    {   // Aqui se define el largo de los array en cada una de las
        // 2 posiciones del vector de array.
        arr2D[i] = std::make_unique<int[]> ( columnas );
        for ( int j = 0; j < 5; j++ ) {
            arr2D[i][j] = j;    // Se accede como un array normal.
            std::cout << arr2D[i][j];
        }
       std::cout << std::endl;
    }
    return 0;
}
```

Referencias

- Una referencia es un *alias* a una variable en memoria.

- Esto se logra mediante la asignación de un ampersand “&”, luego del tipo de variable:

```
nombreTipo x = <debe ser inicializada>;
nombreTipo &y = x;
```

- No confundir con acceder la dirección de memoria de una variable.

- Si se asignan valores a cualquiera de estas variables (x e y), los dos verán reflejados el cambio.

- Se puede ver como dos variable que acceden a la misma posición de memoria.

Referencias (cont'd)

- Las referencias pueden hacerse como en el código a la izquierda, o con el calificador *const* como en el código de la derecha:

```
#include <iostream>

int main(){
    int x = 3; int &y = x;

    std::cout << "x = " << x << ", y = "
    << y << std::endl;   // Imprime "x=3, y=3"
    x = 4;
    std::cout << "x = " << x << ", y = "
    << y << std::endl;   // Imprime "x=4, y=4"
    y = 5;
    std::cout << "x = " << x << ", y = "
                << y << std::endl;   // Imprime "x=5, y=5"

    return 0;
}
```

```
#include <iostream>

int main(){
    int x = 3;
    const int &y = x; // Referencia constante
    // Imprime x = 3 e y = 3
    std::cout << "x = " << x << ", y = " << y << std::endl;
    x = 4;
    // Imprime x = 4 e y = 4
    std::cout << "x = " << x << ", y = " << y << std::endl;
    y = 5;      // Error de compilación
    std::cout << "x = " << x << ", y = " << y << std::endl;
    return 0;
}
```

Pasaje de Parámetros a Funciones

Universidad de SanAndrés

Los pasajes de parámetros pueden ser:

1. Pasaje por valor: copia el valor del parámetro para pasarlo a la función:

```
int square(int value) {return value*value;};
squareValue = square(value);
```

2. Pasaje por referencia: se pasa la variable a la función declarando un alias:

```
void square(int& val) { val *= val; };
square(value);
```

3. Pasaje por puntero: se pasa a la función la dirección de la variable:

```
void square(int* val) { *val  *= *val; };
square(&value);
```

Cuando usar cada una?

1. Pasaje por valor:
   - Cuando no son necesarias modificaciones en las variables.
   - Cuando la cantidad de variables a pasar es chica

2. Pasaje por referencia o por puntero:
   - Cuando se necesitan retornar múltiples valores.
   - Cuando la performance es importante (para no utilizar tanta memoria en la stack al llamar la función).
   - Cuando una modificación en los parámetros a pasar es necesaria.
   - Tener en cuenta que un puntero a una clase/struct usa el operador flecha “->”, mientras que una referencia utilizará el operador punto “.”

Pasaje de Smart Pointers – *unique_ptr*

Un *unique_ptr*, se puede pasas por valor (izquierda) o por referencia (derecha)

```
#include <iostream>
#include <memory>

void myFunc(std::unique_ptr<int> ptr) {
    std::cout << “Valor en la función: " << *ptr << std::endl;
    // `ptr` se va fuera de scope y se libera
}

int main() {
    std::unique_ptr<int> ptr = std::make_unique<int>(5);
    // std::move transfiere el ownership a la función
    myFunc(std::move(ptr));
    // Luego de myFunc, el puntero es null
    if (!ptr)
        std::cout << “Ahora ptr es null." << std::endl;

    return 0;
}
```

Pasando unique_ptr por valor

```
#include <iostream>
#include <memory>

void myFunc(std::unique_ptr<int>& ptr) {
    *ptr = 42;  // Modifica el dato al cual ptr apunta
    std::cout << “Dentro de myFunc: " << *ptr << std::endl;
}

int main() {
    std::unique_ptr<int> ptr = std::make_unique<int>(5);
    std::cout << "Antes de myFunc : " << *ptr << std::endl;
    // Pasa por referencia para evitar transferir ownership
    myFunc(ptr);
    std::cout << “Fuera de myFunc: " << *ptr << std::endl;
    return 0;
}
```

Pasando unique_ptr por referencia

Retornando un *unique_ptr*

Como no pueden existir dos instancias de un *unique_ptr*, es importante que al retornarlos se transfiera el ownership del puntero. Esto se logra con *std::move.*

En el caso de arriba, se permite devolver el puntero directamente. Esto se debe a que es una variable local de la función y el compilador se encarga de hacer el cambio de ownership de *arr* en forma implícita. Nótese que si se retorna *std::move(arr)* también funcionará.

En el segundo caso, si sólo se retorna *arr* sin *std::move*, se estaría intentando copiar el *unique_ptr*, lo cual generaría un error de compilación. Aquí si es necesario usar *std::move* para transferir el ownership del puntero.

```
std::unique_ptr<int[]> allocateArray(int size) {
   std::unique_ptr<int[]> arr = std::make_unique<int[]>(size);
   for (int i = 0; i < size; ++i)
      arr[i] = i + 1;  // Initializa el array con valores de 1 a size
   return arr;        // Retorna el unique_ptr
}

std::unique_ptr<int[]> modifyArray(std::unique_ptr<int[]>
&arr, int size) {
   for (int i = 0; i < size; ++i)
      arr[i] *= 2;                // Procesa los datos del array
   return std::move(arr);   // Transfiere ownership
}
```

Pasaje de Smart Pointers – *shared_ptr*

Universidad de SanAndrés

Un *shared_ptr*, se puede pasas por valor (izquierda) o por referencia (derecha)

```
#include <iostream>
#include <memory>
// Por valor, ambos ptr apuntan a la misma dirección
void myFunc(std::shared_ptr<int> ptr) {
   *ptr = 6;     // Cambiar el valor se vera reflejado en main()
   std::cout << "Count en myFunc: " << ptr.use_count() <<
   std::endl;   // Por referencia el count se incrementa a 2.
}

int main() {
   std::shared_ptr<int> ptr = std::make_shared<int>(5);
   // Pasar por valor incrementa el count, se hace una copia.
   myFunc(ptr);
   std::cout << "Count en ptr: " << ptr.use_count()
   << std::endl;     // Imprime 1, el otro se libero,
   std::cout << “Valor luego de usar la funcion: " << *ptr <<
   std::endl;          // Imprime 6
    return 0;
}
```

Pasando shared_ptr por valor

```
#include <iostream>
#include <memory>

void myFunc(const std::shared_ptr<int>& ptr) {
   *ptr = 6;  // Como es un alias, el valor también cambia en main()
   std::cout << "Count en myFunc: " << ptr.use_count() << std::endl;
   // El count de la referencia no cambia, imprime 1.
}

int main() {
   std::shared_ptr<int> ptr = std::make_shared<int>(5);
   // Pasando por refecencia no modifica el use_count al retornar,
   myFunc(ptr);
   std::cout << "Count en ptr: " << ptr.use_count()
    << std::endl;   // Imprime 1
   std::cout << "Valor luego de usar la funcion: " << *ptr <<
   std::endl;        // Imprime 6
   return 0;
}
```

Pasando shared_ptr por referencia

Punteros a Funciones

Un puntero a una función se declara de la siguiente forma:

*tipoARetornar (*nombrePuntero) (tipoArg1, tipoArg2, …, tipoArgN)*

Declaración de punteros a funciones

```
#include <iostream>

int foo(){}; int goo(){}; double hoo(int){};

int main(){
   int (*fncPtr1)(){&foo};    // puntero a foo, retorna int
   int (*fncPtr2)() = &foo;   // puntero a foo, retorna int
   fncPtr1 = &goo;   // cambia el puntero a goo
   double (*fncPtr3)(int) = &hoo; // puntero a hoo (con
parámetro)
   fncPtr3 = &foo;    // Error por tipos.

   float (*fncPtr4)(){nullptr}; // puntero a null pointer
   float (*fncPtr5)() = NULL;  // puntero a null pointer

   return 0;
}
```

Llamadas a funciones con punteros a funciones

```
#include <iostream>

int square(int x){ return x*x;}

int main(){
   // Inicializar puntero a función
   int (*fncPtr)(int) = &square;
   // Llamar la función con fncPtr:
   (*fncPtr)(5);  // Desreferenciación explícita
   fncPtr(5);      // Desreferenciación implícita
   // Imprime las direcciones de las funciones (son las mismas)
   std::cout << reinterpret_cast<void*>(fncPtr) << std::endl;
   std::cout << reinterpret_cast<void*>(square) << std::endl;
   // recordar: square es una dirección, la función es square()
   return 0;
}
```

Callback Functions

- Una callback function es un puntero a una función que se pasa a una función como parte de sus argumentos.
- Usualmente se utilizan para agregar distintas funcionalidades a una función: como por ejemplo, permitir distintas forma de ordenar un vector (ascendente, descendente, números pares primero, etc.)
- También se usan para evaluar distintas funciones o realizar operaciones en una función dada. Como ejemplo del segundo caso, se puede ver como se pasa una función ($\mathrm{f(x)} = x^2$) a integrar con el método del trapezoide:

$$\int_a^b f(x)dx = h\left[\frac{f(a)+f(b)}{2} + \sum_{k=1}^{n-1} f(a+kh)\right], \qquad h = \frac{b-a}{n}$$

Donde n es el número de intervalos que se usan para aproximar la integral.

Callback Functions para Agregar Funcionalidad

```
#include <iostream>

// Callback funtion para sumar dos números
int sumar(int a, int b) {
   return a + b;
}

// Callback function para logear resultados
void logResultado(int result) {
   std::cout << “El resultado es: " << result << std::endl;
}

// Funcion que acepta callbacks para definir y agregar funcionalidad
int realizarOperacion(int (*operacion)(int, int), int a, int b, void
(*logCallback)(int)) {
   // Realiza la operacion (add)
   int result = operacion(a, b);
   // Llama la callback para logear resultados
   logCallback(result);
   return result;
}
```

```
int main() {
   int a = 5, b = 3;
   // Pasa la funciones "sumar" y "logResultado"
   int result = realizarOperacion(sumar, a, b, logResultado);
   std::cout << "Resultado de la operacion: " << result <<
   std::endl;

   return 0;
}
```

Callback Functions para Aplicar Funcionalidad

```cpp
#include <iostream>

// Regla trapezoidal para la integracion numerica
double trapezoidalRule(double (*func)(double), double a,
double b, int n) {
   // Tamaño del paso
   double h = (b - a) / n;

   // Calcula la suma usando la regla trapezoidal
   double sum = (func(a) + func(b)) / 2.0;  // f(a) + f(b)

   // Suma los terminos intermedios, f(x_i), donde i = 1 a n-1
   for (int i = 1; i < n; ++i) {
      double x = a + i * h;
      sum += func(x);  // f(x_i)
   }

   // Multiplicar por el paso para obtener la aprox a la integral
   return sum * h;
}
```

```cpp
// Function de ejemplo: f(x) = x^2
double squareFunction(double x) {
   return x * x;
}

int main() {
   // Limites de integracion
   double a = 0.0;  // Limite inicial
   double b = 2.0;  // Limite final
   int n = 1000;     // Cantidad de intervalos

   // Resuelve la integral de la callback function
   double result = trapezoidalRule(squareFunction, a, b, n);

   std::cout << "La integral approximada de x^2 desde " << a
         << " a " << b << " es: " << result << std::endl;

   return 0;
}
```

Callback Functions Resumen

- Permiten pasar comportamiento como argumento.
- El código es más flexible y genérico.
- Se prefiere el uso de lambdas en lugar de punteros a funciones.

La clase template std::function

- Una formas más flexible y potente de crear un puntero a una función es mediante la clase template *std::function* de <functional>.
- La mayor flexibilidad se observa con funciones lambda y functors.
- Es importante observar en el código el uso de la palabra reservada *using* para crear un alias y hacer más legible el código y no tan verborrágico.

```
#include <iostream>
#include <functional>

// Para hacer más legible el código se puede usar un alias
//del tipo de variable.
using fcnPtrAlias = std::function<double(int, int)>;

double distance(int x, int y){
   return x*x + y*y + 3.5;
}

int main(){
   fcnPtrAlias fncPtr;
   fncPtr = &distance;
   // Imprime 16.5
   std::cout << fncPtr(2,3) << std::endl;

   return 0;
}
```

Funciones Lambda

Universidad de SanAndrés

- Funciones lambda fueron introducidas en C++11 y estas implementan lo que en ciencias de la computación se conoce como *closures*.
- Un closure es una función tipo objeto (un functor) que puede capturar variables fuera del scope de la función.

La sintaxis de las funciones lambda en C++ son:

Función lambda con captura, parámetros y tipo de retorno:
[capturas] (parámetros) -> tipoARetornar {cuerpo de la función}

Función lambda con parámetros y tipo de retorno:
[] (parámetros) -> tipoARetornar {cuerpo de la función}

Función lambda sólo con tipo de retorno:
[] () -> tipoARetornar {cuerpo de la función}

Función lambda sin captura, sin parámetros y sin tipo de retorno:
[] () -> {cuerpo de la función}

Funciones Lambda - Ejemplos

```
#include <iostream>

int main() {
   // 1. Función lambda con captura, parámetros y tipo de retorno opcional
   int a = 5, b = 3;
   auto lambda1 = [a, b](int x, int y) -> int {
      return a + b + x + y;
   };
   std::cout << "Resultado lambda1: " << lambda1(2, 4) << std::endl;

   // 2. Función lambda con parámetros y tipo de retorno opcional
   auto lambda2 = [](int x, int y) -> int {
      return x * y;
   };
   std::cout << "Resultado lambda2: " << lambda2(2, 4) << std::endl;

   // 3. Función lambda con tipo de retorno opcional (devuelve "true" o "false")
   auto lambda3 = []() -> bool {
      std::cout << "Estableciendo comunicacion con lambda3" << std::endl;
      return true;
   };
   std::cout << std::boolalpha;// std::noboolalpha
   bool result3 = lambda3();
   std::cout << "Resultado lambda3: " << result3 << std::endl;

   // 4. Función lambda sin captura, parámetros y tipo de retorno opcional
   auto lambda4 = []() {
      std::cout << "Mensaje desde lambda4" << std::endl;
   };
   lambda4();    // Imprime: Mensaje desde lambda4

   return 0;
}
```

Funciones Lambda - Ejemplos

Existen otras maneras en que las funciones lambda capturan variables:

- [=] - todas las variables en el scope de la función lambda (excepto las variables static, extern o thread_local) son capturadas por valor. Es decir son de sólo lectura. [&b] captura b como mutable o modificable.
- [&] - todas las variables en el scope de la función lambda (excepto las variables static, extern o thread_local) son capturadas por referencia. Las captura como modificables.

```
#include <iostream>

int main() {
   int a = 5;       // Variable a capturar
   int b = 10;      // Variable a capturar

   // Funcion lambda capturando variables por valor
   auto printSum = [=, &b]() { // función anónima
      b += a;   // a es inmutable, const
      std::cout << “La suma de a y b es: " << b << std::endl;
   };

   printSum();  // Imprime: La suma de a y b es: 15
   std::cout << “El valor de b es: " << b << std::endl;
   // Imprime: El valor de b es: 15
   return 0;
}
```

Funciones Lambda - Ejemplos

En C++ también se pueden crear funciones en forma dinámica.

Es utilizado frecuentemente en el paradigma llamado Programación Funcional.

La función *multiplicar*, crea funciones en tiempo de ejecución, en función del parámetro *factor* pasado.

```
#include <iostream>
#include <functional>

std::function<int(int)> multiplicar(int factor) {
   return [factor](int n) { return n * factor; };
} // Devuelve una funcion lambda

int main() {
  // 'dobleFunc' es una funcion, donde factor=2
   auto dobleFunc = multiplicar(2);
  // 'tripleFunc' es una funcion, donde factor=3
   auto tripleFunc = multiplicar(3);
   std::cout << dobleFunc(5) << std::endl;  // Imprime: 10
   std::cout << tripleFunc(5) << std::endl;  // Imprime: 15
   return 0;
}
```

std::function<int(int)>

Tipo de dato a devolver

Tipo de dato del parámetro

Preguntas?